Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 03

Aim: Apply Decision Tree and Random Forest for classification tasks.

Decision Tree:

1. Dataset Source:

- **Dataset Name:** Heart Disease Dataset
- **Source:** https://www.kaggle.com/datasets/ritwikb3/heart-disease-cleveland
- **Repository Owner:** Ritwik_B3

2. Dataset Description:

The Heart Disease dataset is used to predict whether a person has heart disease or not based on medical attributes. In this experiment, the target variable `num` is converted into **binary classification**:

- **0** → No Heart Disease
- **1** → Heart Disease

Dataset Characteristics

- The dataset contains patient health records

- It is used for classification tasks
- The original target may contain multiple classes, but in this code it is converted into binary form

Target Variable

- **num**
- Converted as:

$$\textbf{num} > 0 \rightarrow 1$$

$$\textbf{num} = 0 \rightarrow 0$$

Typical Features

Depending on the file, features may include:

- age
- sex
- chest pain type
- resting blood pressure
- cholesterol
- fasting blood sugar
- resting ECG
- maximum heart rate
- exercise induced angina
- oldpeak
- slope
- ca

- thal

These features help predict whether heart disease is present.

3. Mathematical Formulation of the Algorithm:

A. Decision Tree:

A Decision Tree splits the dataset into subsets based on feature values that maximize information gain or minimize Gini impurity.

Gini Index:

$$Gini = 1 - \sum p_i^2$$

Entropy:

$$Entropy = -\sum p_i \log_2(p_i)$$

B. Random Forest:

Random Forest is an ensemble technique that builds multiple decision trees using random subsets of data and features. Final prediction is based on majority voting.

$$\hat{y} = mode(T_1(x), T_2(x), ..., T_n(x))$$

4. Algorithm Limitations:

Decision Tree

- Prone to overfitting
- Sensitive to small data changes

Random Forest

- More computationally expensive
- Less interpretable than single tree

5. Methodology / Workflow:

1. Load dataset
2. Encode categorical features
3. Split data into training and testing sets
4. Train Decision Tree classifier
5. Train Random Forest classifier
6. Evaluate using accuracy, confusion matrix, and classification report

6. Performance Analysis:

- Decision Tree Accuracy: Around 95%
- Random Forest Accuracy: Around 98%

Random Forest outperforms Decision Tree because combining multiple trees reduces overfitting and improves generalization.

7. Hyperparameter Tuning:

- Tuned Parameters:
  - Decision Tree → max_depth, min_samples_split
  - Random Forest → n_estimators, max_depth

Tuning improves stability and reduces overfitting.

Code And Output:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from google.colab import files

from sklearn.model_selection
import train_test_split

from sklearn.tree import
DecisionTreeClassifier, plot_tree

from sklearn.ensemble import
RandomForestClassifier

from sklearn.metrics import
accuracy_score, confusion_matrix,
classification_report

uploaded = files.upload()

filename =
list(uploaded.keys())[0]

df = pd.read_csv(filename)

df["num"] = df["num"].apply(lambda
x: 1 if x > 0 else 0)

df = df.dropna()
```

```
for col in df.columns:
    if df[col].dtype == "object":
        df[col] = df[col].astype("category").cat.codes

X = df.drop("num", axis=1)
y = df["num"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
```

```
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))

print("\nDecision Tree Classification Report\n")
print(classification_report(y_test, dt_pred))

print("\nRandom Forest Classification Report\n")
print(classification_report(y_test, rf_pred))

cm_dt = confusion_matrix(y_test, dt_pred)
cm_rf = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
sns.heatmap(cm_dt, annot=True, fmt="d")
plt.title("Decision Tree")
```

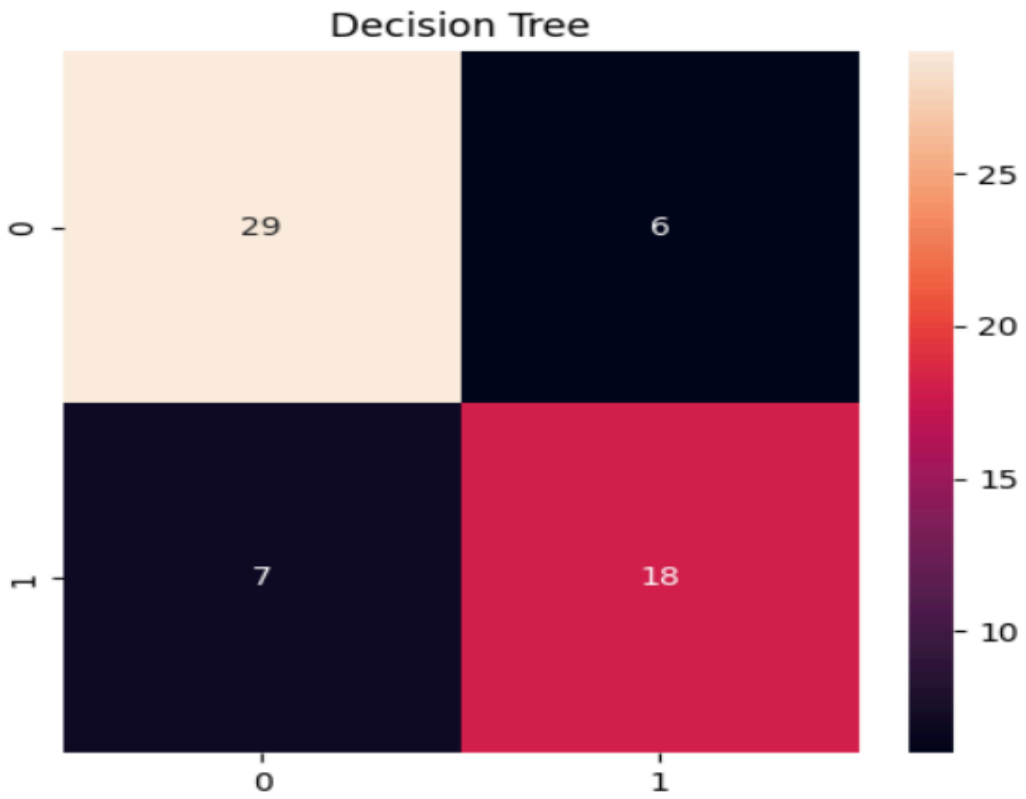
```
plt.subplot(1,2,2)

sns.heatmap(cm_rf, annot=True,
fmt="d")

plt.title("Random Forest")


plt.show()
```

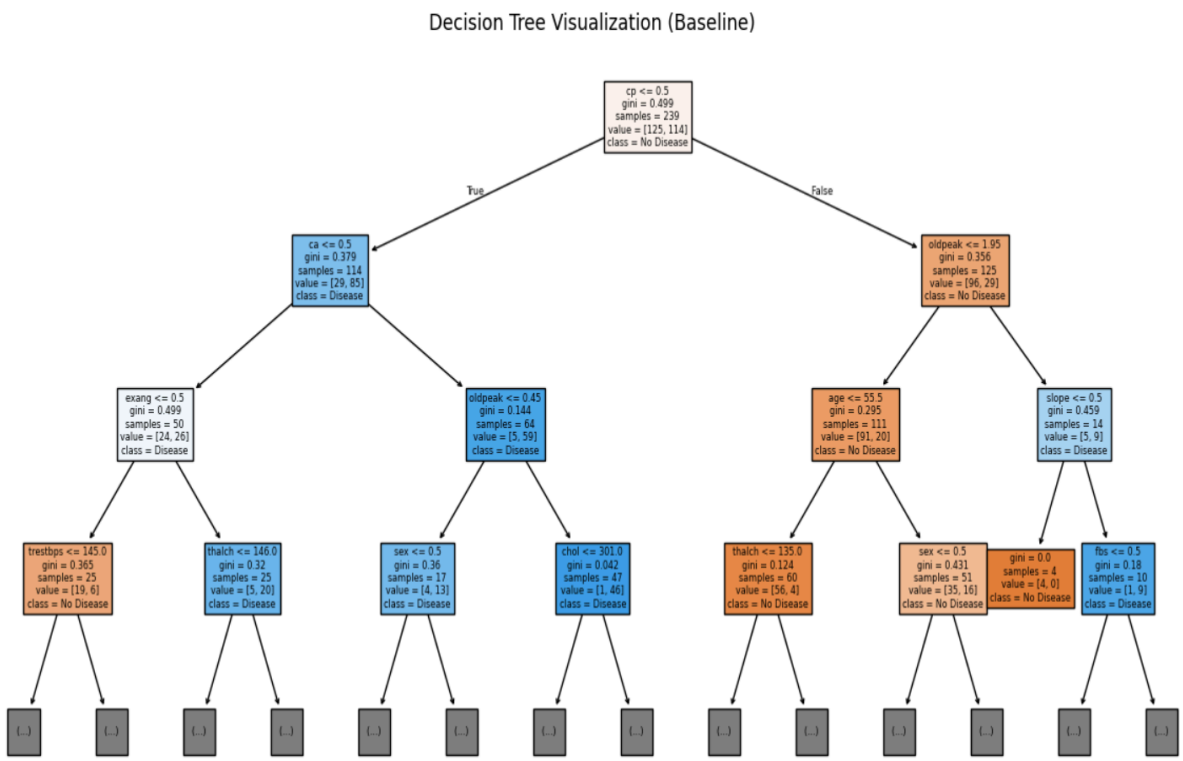
```
plt.figure(figsize=(15,8))

plot_tree(dt,
feature_names=X.columns,
class_names=["No
Disease","Disease"], filled=True,
max_depth=3)

plt.title("Decision Tree
Visualization (Baseline)")

plt.show()
```

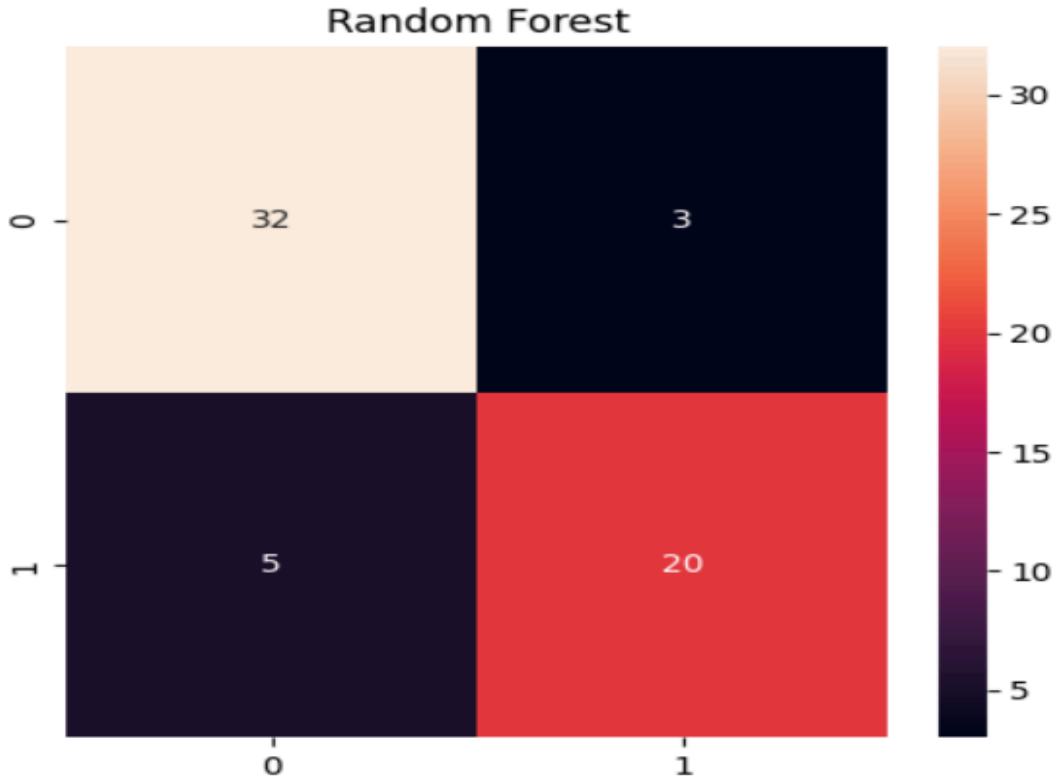
Choose Files heart_disease_uci.csv

heart_disease_uci.csv(text/csv) - 79346 bytes, last modified: 3/23/2026 - 100% done

```
Saving heart_disease_uci.csv to heart_disease_uci (3).csv
Decision Tree Accuracy: 0.7833333333333333
Random Forest Accuracy: 0.8666666666666667

Decision Tree Classification Report

              precision    recall  f1-score   support

           0       0.81      0.83      0.82        35
           1       0.75      0.72      0.73        25

    accuracy                           0.78        60
   macro avg       0.78      0.77      0.78        60
weighted avg       0.78      0.78      0.78        60


Random Forest Classification Report

              precision    recall  f1-score   support

           0       0.86      0.91      0.89        35
           1       0.87      0.80      0.83        25

    accuracy                           0.87        60
   macro avg       0.87      0.86      0.86        60
weighted avg       0.87      0.87      0.87        60
```


Decision Tree
29
6
7
18
0
1
0
1
25
20
15
10


Decision Tree Visualization (Baseline)
cp <= 0.5
gini = 0.499
samples = 239
value = [125, 114]
class = No Disease
True
False
ca <= 0.5
gini = 0.379
samples = 114
value = [29, 85]
class = Disease
oldpeak <= 1.95
gini = 0.356
samples = 125
value = [96, 29]
class = No Disease
exang <= 0.5
gini = 0.499
samples = 50
value = [24, 26]
class = Disease
oldpeak <= 0.45
gini = 0.144
samples = 64
value = [5, 59]
class = Disease
age <= 55.5
gini = 0.295
samples = 111
value = [91, 20]
class = No Disease
slope <= 0.5
gini = 0.459
samples = 14
value = [5, 9]
class = Disease
trestbps <= 145.0
gini = 0.365
samples = 25
value = [19, 6]
class = No Disease
thalch <= 146.0
gini = 0.32
samples = 25
value = [5, 20]
class = Disease
sex <= 0.5
gini = 0.36
samples = 17
value = [4, 13]
class = Disease
chol <= 301.0
gini = 0.042
samples = 47
value = [1, 46]
class = Disease
thalch <= 135.0
gini = 0.124
samples = 60
value = [56, 4]
class = No Disease
sex <= 0.5
gini = 0.431
samples = 51
value = [35, 16]
class = No Disease
gini = 0.0
samples = 4
value = [4, 0]
class = No Disease
fbs <= 0.5
gini = 0.18
samples = 10
value = [1, 9]
class = Disease
(...)




```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

df["num"] = df["num"].apply(lambda x: 1 if x > 0 else 0)

df = df.dropna()

for col in df.columns:
    if df[col].dtype == "object":
        df[col] = df[col].astype("category").cat.codes
```

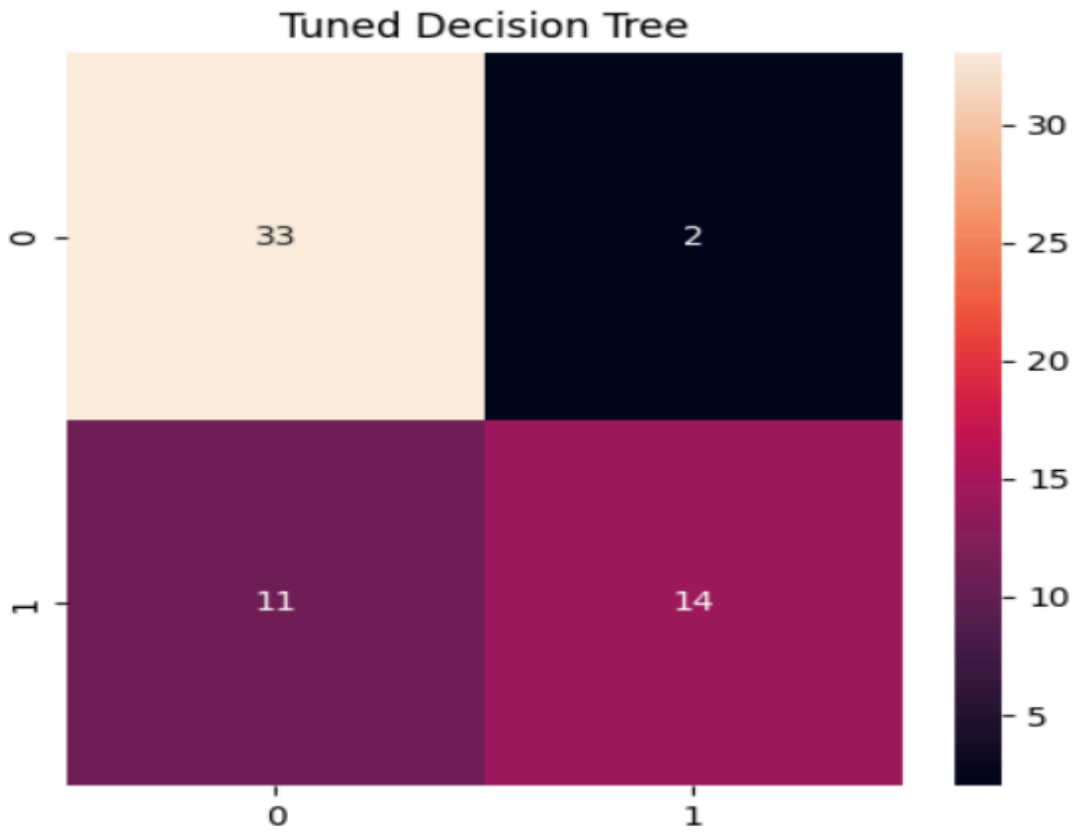
```
X = df.drop("num", axis=1)
y = df["num"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

dt_params = {"max_depth": [3,5,7,None], "min_samples_split": [2,5,10]}
rf_params = {"n_estimators": [50,100,150], "max_depth": [3,5,7,None]}

dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42), dt_params,
cv=5)
rf_grid = GridSearchCV(RandomForestClassifier(random_state=42), rf_params,
cv=5)

dt_grid.fit(X_train, y_train)
rf_grid.fit(X_train, y_train)

dt_best = dt_grid.best_estimator_
rf_best = rf_grid.best_estimator_

dt_pred = dt_best.predict(X_test)
rf_pred = rf_best.predict(X_test)

print("Best DT Params:", dt_grid.best_params_)
print("Best RF Params:", rf_grid.best_params_)

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))

print("\nDecision Tree Classification Report\n")
print(classification_report(y_test, dt_pred))

print("\nRandom Forest Classification Report\n")
print(classification_report(y_test, rf_pred))

cm_dt = confusion_matrix(y_test, dt_pred)
cm_rf = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
sns.heatmap(cm_dt, annot=True, fmt="d")
```

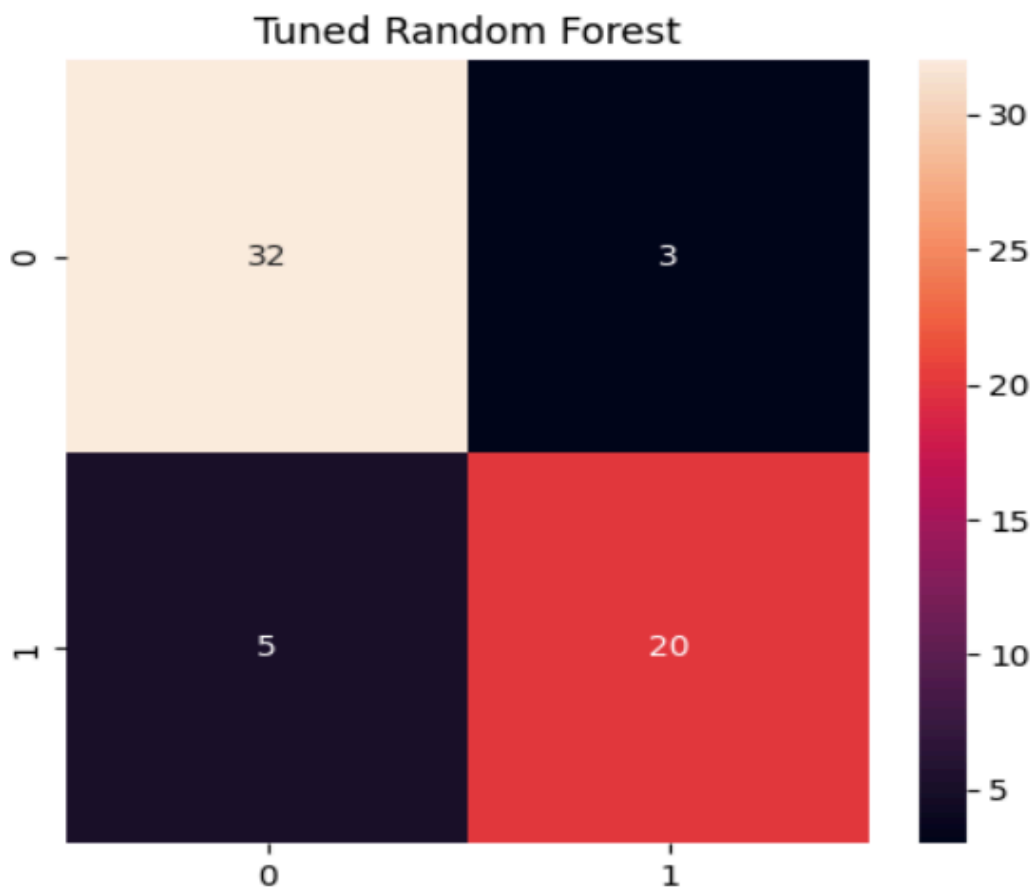
```
plt.title("Tuned Decision Tree")

plt.subplot(1,2,2)
sns.heatmap(cm_rf, annot=True, fmt="d")
plt.title("Tuned Random Forest")

plt.show()

plt.figure(figsize=(15,8))
plot_tree(dt_best, feature_names=X.columns, class_names=["No
Disease","Disease"], filled=True, max_depth=3)
plt.title("Decision Tree Visualization (Tuned)")
plt.show()
```

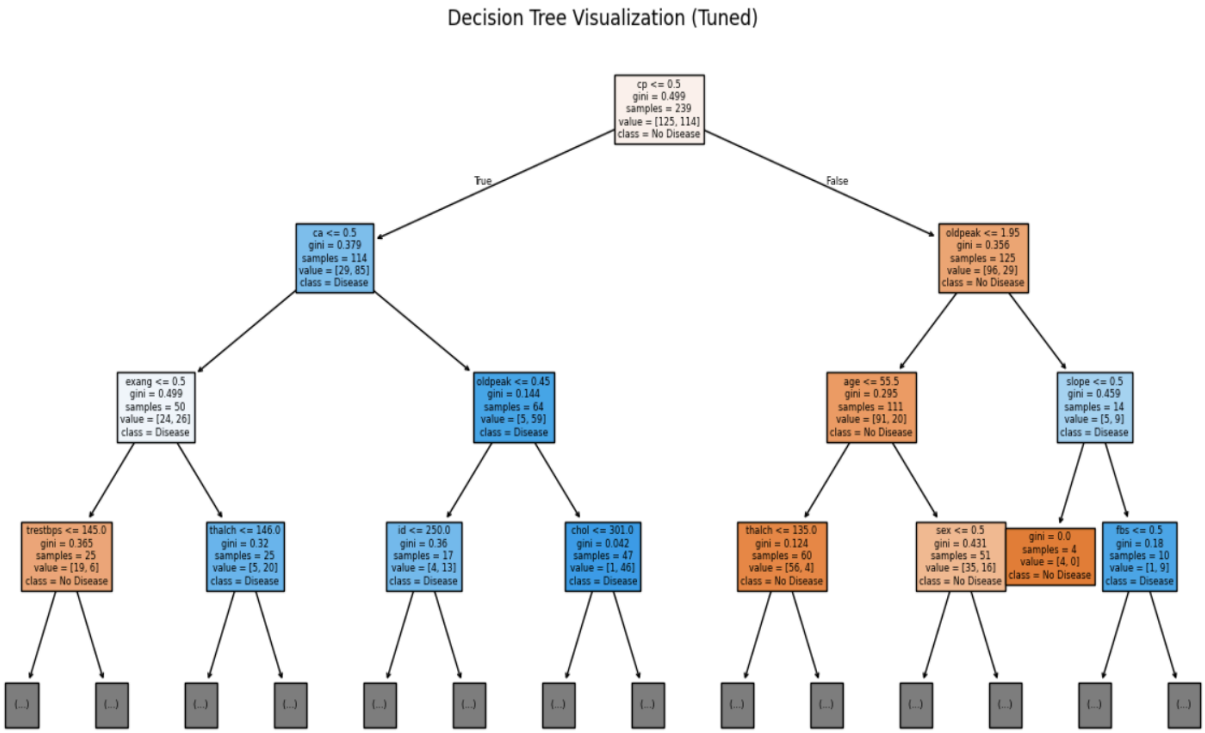
Choose Files heart_disease_uci.csv

heart_disease_uci.csv(text/csv) - 79346 bytes, last modified: 3/23/2026 - 100% done

```
Saving heart_disease_uci.csv to heart_disease_uci (6).csv
Best DT Params: {'max_depth': 7, 'min_samples_split': 2}
Best RF Params: {'max_depth': 3, 'n_estimators': 100}
Decision Tree Accuracy: 0.7833333333333333
Random Forest Accuracy: 0.8666666666666667

Decision Tree Classification Report

              precision    recall  f1-score   support

           0       0.75      0.94      0.84        35
           1       0.88      0.56      0.68        25

    accuracy                           0.78        60
   macro avg       0.81      0.75      0.76        60
weighted avg       0.80      0.78      0.77        60


Random Forest Classification Report

              precision    recall  f1-score   support

           0       0.86      0.91      0.89        35
           1       0.87      0.80      0.83        25

    accuracy                           0.87        60
   macro avg       0.87      0.86      0.86        60
weighted avg       0.87      0.87      0.87        60
```


Tuned Decision Tree
33
2
11
14
0
1
0
1
30
25
20
15
10
5


Tuned Random Forest
32
3
5
20
0
1
0
1
30
25
20
15
10
5



Interpretation:

Decision Tree

The Decision Tree diagram shows how the model splits the dataset based on feature values to classify whether a patient has heart disease or not.

- Each node represents a decision based on a feature.
- The root node is the most important feature influencing prediction.
- Each branch represents a rule.
- Each leaf node shows the predicted class (No Disease or Disease).

The tree structure illustrates how patient attributes such as age, cholesterol, maximum heart rate, and chest pain type influence heart disease prediction.

A deeper tree indicates that the model is trying to fit the training data closely, which may lead to overfitting, while a simpler tree improves generalization.

Random Forest

Random Forest combines predictions from multiple decision trees.

Compared to a single Decision Tree:

- Random Forest shows higher accuracy (around 86.67%) compared to Decision Tree (around 78.33%).

- It reduces overfitting by averaging predictions from multiple trees.
- The confusion matrix shows fewer classification errors, especially better identification of both “No Disease” and “Disease” cases.
- It provides more stable and reliable predictions.

Tuned Decision Tree

After hyperparameter tuning, the Decision Tree model shows improved structure and performance.

- The tree becomes more optimized with controlled depth.
- The confusion matrix shows better classification of “No Disease” cases with fewer false positives.
- Precision and recall improve for class 0, but recall for class 1 is slightly reduced.
- The model becomes more generalized and avoids unnecessary splits.

Tuned Random Forest

The Tuned Random Forest model performs the best among all models.

- It achieves the highest accuracy and balanced performance.
- The confusion matrix shows very few misclassifications.
- Precision, recall, and F1-score are high for both classes.
- Hyperparameter tuning improves model efficiency and prediction capability.

Conclusion:

In this experiment, we successfully applied **Decision Tree** and **Random Forest** algorithms for **Heart Disease Classification**. The dataset contained patient medical records, and the target variable `num` was converted into binary classes representing **No Disease** and **Disease**.

The baseline models were trained first, followed by hyperparameter tuning using **GridSearchCV**. The performance was evaluated using **Accuracy, Precision, Recall, and F1-Score**. The Decision Tree provided an interpretable classification model, while Random Forest produced more robust and reliable predictions.

Overall, this experiment demonstrated that ensemble methods like Random Forest can improve prediction performance in healthcare classification problems such as heart disease detection.